ABSTRACT:

This project presents the design, implementation, and verification of a parameterizable Hamming Code Single Error Correction (SEC) Error-Correcting Code (ECC) system using SystemVerilog. The hardware architecture comprises a transmitter-side encoder that computes and inserts even-parity bits into power-of-two positions, and a receiver-side decoder that generates a syndrome vector to detect and correct single-bit transmission errors while recovering the original data word. To ensure complete functional verification, a modular, class-based, layered testbench architecture is constructed using SystemVerilog OOP constructs. The testbench comprises a generator, driver, monitor, and self-checking scoreboard interconnected via transaction mailboxes and an interface bundle. Randomized stimuli and deliberate single-bit error injections are applied to model noisy channel conditions. Functional simulation and waveform analysis, executed using the Cadence Xcelium simulator and SimVision debugging platform, validate that 100% of injected single-bit errors are correctly located and corrected, demonstrating robust hardware reliability and protocol compliance.

PROBLEM STATEMENT:

The project focuses on the

Design and Verification of a Parameterizable Hamming Code Single Error Correction (SEC) Error-Correcting Code (ECC) System using SystemVerilog.

Digital data can be corrupted due to noise during transmission or storage. The main problem is to detect and correct a single-bit error in the received data without retransmission.

The proposed system encodes the input data by adding parity bits and, at the receiver, calculates the syndrome to identify the error position, correct the erroneous bit, and recover the original data. The design is verified using a modular SystemVerilog testbench with randomized data and single-bit error injection.

Example of Hamming Code – Single-Bit Error Correction

Consider a Hamming (7,4) code, where 4 data bits are protected using 3 parity bits.

Let the data be:

$$\text{Data} = 1011$$

The 7-bit Hamming code positions are arranged as:

Position	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Bit	P_1	P_2	D_1	P_4	D_2	D_3	D_4
Value	P_1	P_2	1	P_4	0	1	1

Using even parity:

- P_1 checks positions 1, 3, 5, 7 → $P_1 = 0$
- P_2 checks positions 2, 3, 6, 7 → $P_2 = 1$
- P_4 checks positions 4, 5, 6, 7 → $P_4 = 0$

Therefore, the transmitted Hamming code is:

0110011

Suppose bit 6 is corrupted during transmission:

Received: 0110001

The receiver calculates the syndrome:

$S_1 = 1, S_2 = 1, S_4 = 1$

Therefore,

Syndrome = $111_2 = 7$

This indicates an error at position 7. The receiver flips bit 7 and obtains:

Corrected code = 0110011

The parity bits are then removed, giving the original data:

Recovered Data = 1011

OBJECTIVES:

The main objectives of the project are:

1. To design a parameterizable Hamming Code Single Error Correction (SEC) ECC system using SystemVerilog.
2. To automatically determine the required number of parity bits based on the selected data width.
3. To encode input data by inserting parity bits at the required positions and generate the Hamming codeword.
4. To calculate the syndrome at the receiver for detecting and locating a single-bit error.
5. To correct the identified single-bit error and recover the original data.
6. To design a modular SystemVerilog testbench for functional verification using randomized data and single-bit error injection.
7. To verify the encoder and decoder operation using Cadence Xcelium.

SYSTEM REQUIREMENTS AND SPECIFICATIONS:

Hardware Requirements

Component	**Specification**
Processor	Intel Core i5 / AMD Ryzen 5 or above
RAM	Minimum 8 GB (16 GB Recommended)
Storage	100 GB Free Disk Space

Operating System	Linux (RHEL/CentOS/Ubuntu)
Display	1920 × 1080 Resolution
Network	LAN or Internet Connection (for License Server)

Software Requirements

Software	**Purpose**
SystemVerilog	RTL Design and Verification
Cadence Xcelium	RTL Simulation and Functional Verification
Cadence SimVision	Waveform Analysis and Debugging
Cadence Genus	Logic Synthesis
Cadence Conformal LEC	Logic Equivalence Checking
Cadence Innovus	Physical Design Implementation
Cadence Tempus	Static Timing Analysis
Cadence Modus	Design for Testability
Cadence PVS	Physical Verification
90 nm Standard Cell Library	ASIC Technology / Standard Cell Library

Design Specifications

Parameter	**Specification**
Project Title	Design and Verification of a Parameterizable Hamming Code Single Error Correction (SEC) ECC System
Design Language	SystemVerilog
ECC Technique	Hamming Code
Error Correction	Single-Bit Error Correction (SEC)
Input Data Width	Parameterizable
Parity Bits	Automatically determined based on data width
Parity Type	Even Parity
Codeword Width	Data Bits + Parity Bits
Parity Bit Positions	1, 2, 4, 8, 16, ...
Arbitration / Coding Method	Hamming SEC Encoding

Error Detection Method	Syndrome Calculation
Error Correction Method	Bit Inversion at Detected Error Position
Shared Channel	Communication / Storage Channel
Verification Method	SystemVerilog Testbench
Error Injection	Single-Bit Error Injection
Test Data	Randomized Data
Clock Type	Positive Edge Triggered *(if implemented as synchronous RTL)*
Reset Type	Synchronous Reset *(if included in RTL)*
Simulation Tool	Cadence Xcelium
Waveform Analysis	Cadence SimVision
Technology	90 nm Technology
Design Methodology	RTL-to-GDSII Flow
ASIC Flow	RTL → Synthesis → LEC → Physical Design → STA → DFT → Physical Verification

Parameterization Specification

The number of parity bits $r$is calculated according to:

$$\boxed{2^r \geq m + r + 1}$$

where $m$is the number of input data bits.

The total Hamming codeword width is:

$$\boxed{n = m + r}$$

PROPOSED ARCHITECTURE:

The proposed system is a parameterizable Hamming Code Single Error Correction (SEC) ECC system implemented using SystemVerilog. The architecture consists of an Hamming Encoder, Error Injection/Communication Channel, Syndrome Generator, Error Correction Unit, and Data Extraction Unit.

The system accepts a configurable number of data bits. Based on the data width, the required number of parity bits is automatically calculated using:

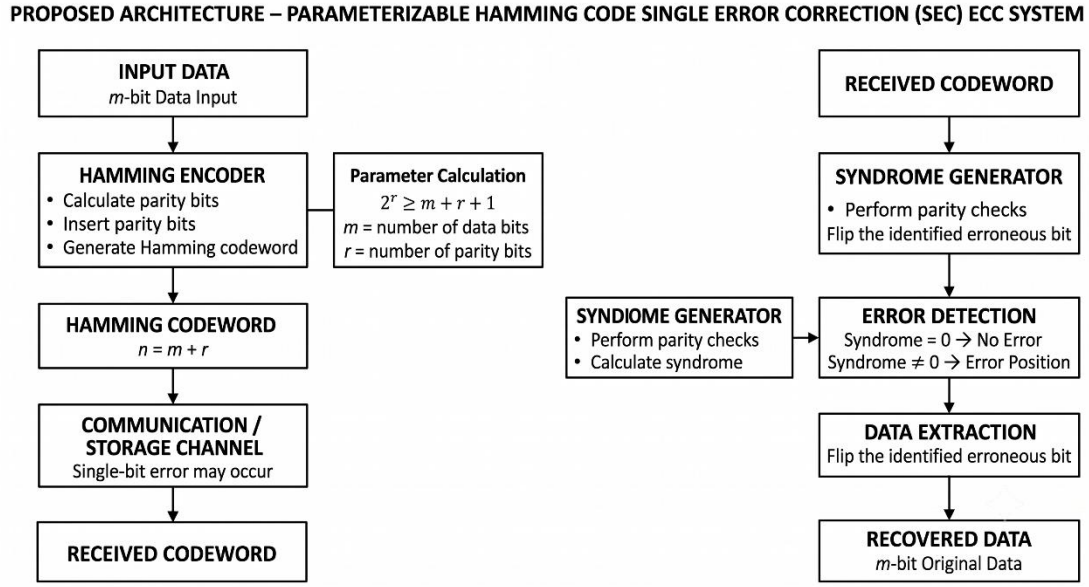
$$2^r \geq m + r + 1$$

where $m$is the number of data bits and $r$is the number of parity bits.

The encoder generates the Hamming codeword by inserting parity bits at positions 1,2,4,8, ... During transmission or storage, a single-bit error may occur. At the receiver, the syndrome is

calculated to identify the error position. If the syndrome is non-zero, the corresponding bit is inverted to correct the error. Finally, the parity bits are removed to recover the original data.

1.Block Diagram



2.Module Description

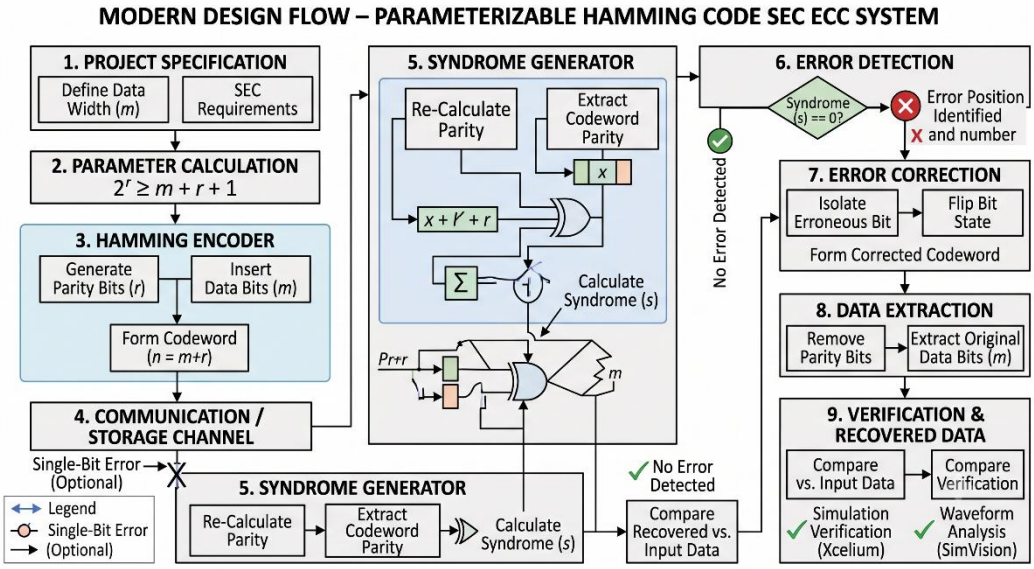
Module	Description
1. Parameter Calculation Module	Determines the required number of parity bits $r$automatically based on the selected data width m, using $2^r \geq m + r + 1$.
2. Hamming Encoder	Accepts the input data and generates the Hamming codeword by placing data bits and parity bits at their appropriate positions.
3. Parity Generator	Calculates the values of the parity bits using even-parity checking groups.
4. Error Injection Module	Used during verification to introduce a single-bit error at different positions of the generated codeword.
5. Syndrome Generator	Performs parity checks on the received codeword and produces the syndrome value representing the position of a single-bit error.
6. Error Detection Module	Examines the syndrome. A zero syndrome indicates no error, while a non-zero syndrome indicates the location of an error.
7. Error Correction Module	Corrects the detected single-bit error by inverting the bit at the position indicated by the syndrome.

8. Data Extraction Module	Removes the parity bits from the corrected codeword and extracts the original data bits.
9. Verification/Testbench Module	Applies different and randomized input data, injects single-bit errors, and verifies that the decoder correctly recovers the original data.
10. Simulation and Waveform Analysis	Cadence Xcelium is used for functional simulation and Cadence SimVision is used to observe and analyze encoder, syndrome, error-correction, and recovered-data waveforms.

DESIGN METHODOLOGY:

The design methodology describes the systematic procedure followed to design, implement, and verify the Parameterizable Hamming Code Single Error Correction (SEC) ECC System using SystemVerilog. The methodology consists of parameter calculation, Hamming encoding, error injection, syndrome generation, error correction, data extraction, and functional verification.

1.Design Flow



Design Flow Description

1. **Project Specification:** The required data width and single-bit error correction requirements are defined.
2. **Parameter Calculation:** The number of parity bits is automatically determined according to:

$$2^r \geq m + r + 1$$

where $m$is the number of data bits and $r$is the number of parity bits.

3. **Hamming Encoding:** The input data bits are placed in the appropriate positions, while parity bits are inserted at positions 1,2,4,8, ... Even parity is then calculated to generate the Hamming codeword.
4. **Error Injection:** During verification, a single-bit error is intentionally introduced at different positions of the generated codeword.
5. **Syndrome Generation:** The receiver performs parity checks on the received codeword and calculates the syndrome value.
6. **Error Detection:** A zero syndrome indicates that no error is detected. A non-zero syndrome identifies the position of a possible single-bit error.
7. **Error Correction:** The bit corresponding to the syndrome value is inverted to correct the detected single-bit error.
8. **Data Extraction:** The parity bits are removed from the corrected codeword and the original data bits are recovered.
9. **Verification:** The recovered data is compared with the original input data to confirm correct operation.
10. **Simulation and Analysis:** Functional simulation is performed using Cadence Xcelium, and the resulting signals and error-correction process are analyzed using Cadence SimVision.

2. Functional Description

The proposed Hamming SEC ECC system performs two main operations: encoding at the transmitter and decoding with error correction at the receiver.

A. Encoding Operation

The Hamming encoder receives an m-bit input data.

The required number of parity bits $r$is automatically calculated using:

$$2^r \geq m + r + 1$$

The parity bits are placed at positions:

$$1,\ 2,\ 4,\ 8,\ 16, \ldots$$

The remaining positions are occupied by the data bits. Even parity is calculated for each parity group, and the complete Hamming codeword is generated.

The total codeword length is:

$$n = m + r$$

Example: For 4-bit input data:

$$2^3 \geq 4 + 3 + 1$$

Therefore, 3 parity bits are required.

$$n = 4 + 3 = 7$$

Hence, the system generates a Hamming (7,4) codeword.

B. Transmission / Storage

The generated Hamming codeword is transmitted through a communication channel or stored in a memory system.

During transmission or storage, noise, interference, or hardware faults can cause a single bit to change.

Original Codeword - Communication / Storage Channel - Single-Bit Error - Received

The received codeword may therefore differ from the original transmitted codeword by one bit.

C. Syndrome Calculation

The decoder receives the possibly corrupted codeword and performs parity checks.

The results of the parity checks form the **syndrome**.

For a Hamming (7,4) code:

$$S = [S_4 \; S_2 \; S_1]$$

The syndrome provides information about the error condition:

- Syndrome = 000: No error detected.
- Syndrome ≠ 000: Single-bit error detected.
- Syndrome value: Indicates the position of the erroneous bit.

D. Error Correction

When a non-zero syndrome is obtained, its binary value represents the position of the erroneous bit.

The decoder corrects the error by inverting the identified bit:

$$\text{Corrected Bit} = \overline{\text{Erroneous Bit}}$$

Thus, the corrupted codeword is converted back into the correct codeword.

E. Data Recovery

After error correction, the parity bits are removed from the corrected codeword. The remaining data bits are extracted to recover the original input data.

Received Codeword - Syndrome Calculation - Error Position - Bit Correction - Corrected Codeword - Remove Parity Bits - Recovered Data

F. Verification

The System Verilog testbench verifies the complete ECC system by:

- Generating different input data patterns.
- Generating the corresponding Hamming codewords.
- Injecting single-bit errors at different codeword positions.
- Calculating and checking the syndrome.
- Verifying the identified error position.
- Checking successful error correction.
- Comparing recovered data with the original input data.
- Observing the encoder, decoder, syndrome, and corrected data waveforms using Cadence Xcelium and SimVision.

RTL DESIGN

1. Module Description

The proposed Hamming Code SEC ECC system is implemented using a parameterizable RTL architecture in SystemVerilog. The RTL design consists of two major functional modules: the Hamming Encoder and the Hamming Decoder.

The Hamming Encoder accepts an m-bit input data word and generates a Hamming codeword by adding the required number of parity bits. The number of parity bits $r$is automatically calculated using the condition:

$$2^r \geq m + r + 1$$

The parity bits are inserted at positions $1,2,4,8,$ ..., while the remaining positions are occupied by data bits. Even parity is used for the generation of the codeword.

The Hamming Decoder receives the transmitted or stored codeword. A single-bit error may be introduced before the codeword reaches the decoder. The decoder performs parity checks and generates a syndrome. A zero syndrome indicates that no error is present, whereas a non-zero syndrome identifies the position of the erroneous bit.

The identified bit is corrected by complementing its value. Finally, the parity bits are removed and the original data is recovered.

Main RTL modules:

Module	**Function**
hamming_encoder	Generates Hamming codeword from input data
hamming_decoder	Detects and corrects a single-bit error
hamming_ecc_top	Integrates encoder and decoder
Testbench	Verifies functionality using error injection

2. RTL Architecture

The RTL architecture is divided into encoding, error injection, decoding, and data recovery stages.

Hamming Encoder

The encoder performs the following operations:

1. Accepts the input data.
2. Calculates the required number of parity bits.
3. Determines the total codeword length.
4. Places parity bits at positions 1,2,4,8, ...
5. Places data bits in the remaining positions, Calculates even parity.
6. Generates the final Hamming codeword.

Error Injection

For verification purposes, a single-bit error is introduced into the generated codeword. The error can be injected at different positions to verify the correction capability of the decoder.

For example:

$$\text{Original Codeword} = 0110011$$

If bit position 6 is corrupted:

$$\text{Received Codeword} = 0110001$$

Hamming Decoder

The decoder performs parity checks on the received codeword. The parity check results are combined to form the syndrome.

For a Hamming (7,4) code:

$$S = S_4 S_2 S_1$$

The syndrome value directly represents the position of the erroneous bit.

Syndrome	Condition
000	No error
001	Error at position 1
010	Error at position 2
011	Error at position 3
100	Error at position 4
101	Error at position 5

110	Error at position 6
111	Error at position 7

If the syndrome is non-zero, the corresponding bit is inverted.

Data Extraction

After correction, the parity positions are removed and the remaining data bits are collected to generate the recovered data.

3. Design Implementation

The RTL implementation is designed to support different data widths. Instead of designing a fixed Hamming (7,4) circuit, the number of parity bits is determined automatically from the selected data width.

For a data width of $m = 4$:

$$2^r \geq m + r + 1$$

For $r = 3$:

$$2^3 \geq 4 + 3 + 1$$
$$8 \geq 8$$

Therefore:

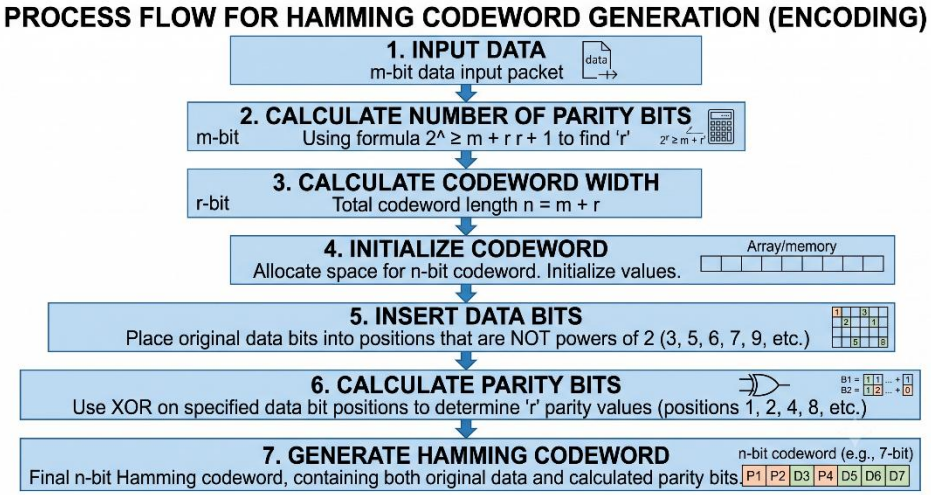
$$r = 3$$

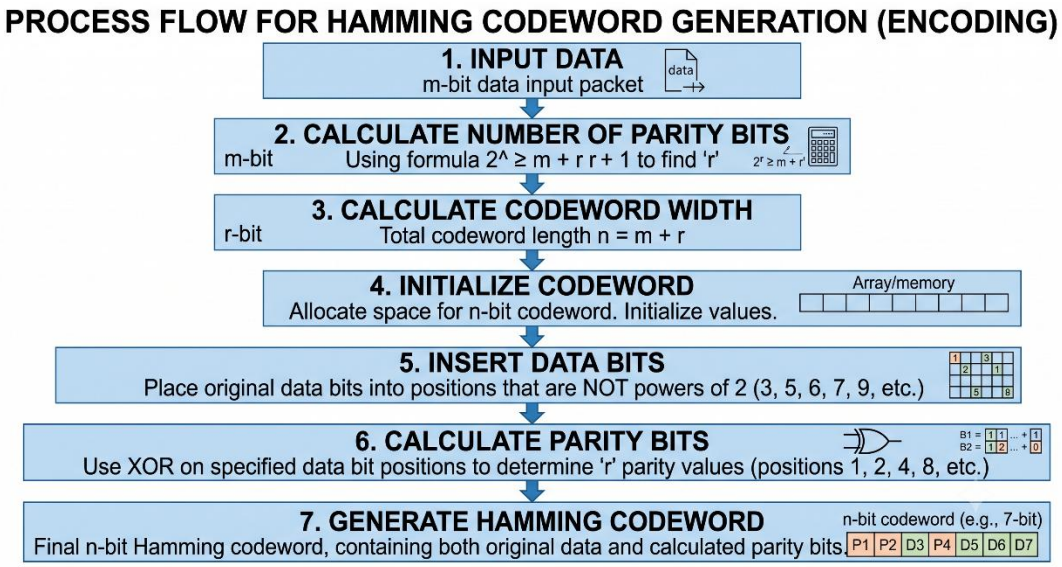
and the total codeword width is:

$$n = m + r = 4 + 3 = 7$$

Thus, the system generates a Hamming (7,4) code.

The RTL implementation uses loops to place data and parity bits. This makes the design scalable and allows the same RTL architecture to be used for different input data widths.





4. RTL Code

Hamming Encoder

```
module hamming_encoder #(
   parameter int DATA_WIDTH = 4)(
   input  logic [DATA_WIDTH-1:0] data_in,
   output logic [CODE_WIDTH-1:0] codeword);
   function automatic int calc_parity_bits(input int m);
      int r;
      begin
         r = 0;
         while ((2**r) < (m + r + 1))
            r = r + 1;
         calc_parity_bits = r;
      end
   endfunction
   localparam int PARITY_BITS = calc_parity_bits(DATA_WIDTH);
   localparam int CODE_WIDTH  = DATA_WIDTH + PARITY_BITS;
   integer i;
   integer j;
   integer data_index;
   integer parity_position;
   logic parity;
```

```
    always_comb begin
        codeword = '0;
        data_index = 0;
        // Insert data bits
        for (i = 1; i <= CODE_WIDTH; i = i + 1) begin
            if ((i != 1) &&
                (i != 2) &&
                (i != 4) &&
                (i != 8) &&
                (i != 16) &&
                (i != 32)) begin
                if (data_index < DATA_WIDTH) begin
                    codeword[i-1] = data_in[data_index];
                    data_index = data_index + 1;
                end
            end
        end
        // Calculate parity bits
        for (j = 0; j < PARITY_BITS; j = j + 1) begin
            parity_position = 2 ** j;
            parity = 1'b0;
            for (i = 1; i <= CODE_WIDTH; i = i + 1) begin
                if ((i & parity_position) != 0)
                    parity = parity ^ codeword[i-1];
            end
            codeword[parity_position-1] = parity;
        end
    end
endmodule
```

Hamming Decoder

```
module hamming_decoder #(
    parameter int DATA_WIDTH = 4)(
```

```
input  logic [CODE_WIDTH-1:0] codeword_in,
output logic [DATA_WIDTH-1:0] data_out,
output logic [PARITY_BITS-1:0] syndrome,
output logic error_detected,
output logic error_corrected);
function automatic int calc_parity_bits(input int m);
   int r;
   begin
      r = 0;
      while ((2**r) < (m + r + 1))
         r = r + 1;
      calc_parity_bits = r;
   end
endfunction
localparam int PARITY_BITS = calc_parity_bits(DATA_WIDTH);
localparam int CODE_WIDTH  = DATA_WIDTH + PARITY_BITS;
logic [CODE_WIDTH-1:0] corrected_codeword;
integer i;
integer j;
integer parity_position;
integer data_index;
logic parity;
always_comb begin
   syndrome = '0;
   corrected_codeword = codeword_in;
   data_out = '0;
   // Syndrome calculation
   for (j = 0; j < PARITY_BITS; j = j + 1) begin
      parity_position = 2 ** j;
      parity = 1'b0;
      for (i = 1; i <= CODE_WIDTH; i = i + 1) begin
         if ((i & parity_position) != 0)
```

```
            parity = parity ^ codeword_in[i-1];
        end
        syndrome[j] = parity;
    end
    // Error detection and correction
    if (syndrome != '0) begin
        error_detected = 1'b1;
        if (syndrome <= CODE_WIDTH) begin
            corrected_codeword[syndrome-1] =  ~corrected_codeword[syndrome-1];
            error_corrected = 1'b1;
        end
    end
    else begin
        error_detected = 1'b0;
        error_corrected = 1'b0;
    end
    // Extract data bits
    data_index = 0;
    for (i = 1; i <= CODE_WIDTH; i = i + 1) begin
        if ((i != 1) &&
            (i != 2) &&
            (i != 4) &&
            (i != 8) &&
            (i != 16) &&
            (i != 32)) begin
            if (data_index < DATA_WIDTH) begin
                data_out[data_index] =
                    corrected_codeword[i-1];
                data_index = data_index + 1;
            end
        end
    end
```

```
    end
endmodule
```

VERIFICATION

1. Testbench Description

The proposed Hamming SEC ECC system is verified using a layered SystemVerilog testbench. The verification environment consists of multiple files, where each file performs a specific function such as parameter definition, signal communication, transaction generation, stimulus driving, monitoring, result checking, and test execution.

The testbench architecture contains the following files:

1. globals.sv – Global parameters
2. interface.sv – DUT communication interface
3. transaction.sv – Transaction data structure
4. generator.sv – Random transaction generation
5. driver.sv – Driving input and error injection
6. monitor.sv – Monitoring DUT signals
7. scoreboard.sv – Result verification
8. pgtest.sv – Testbench environment
9. hamming_tb.sv – Top-level layered testbench
10. tb_hamming_encoder.sv – Directed testbench

Testbench File Description

1. Global Configuration – globals.sv

The globals.sv file contains common parameters used throughout the verification environment. These parameters define the data width, number of parity bits, and total Hamming codeword width.

The main parameters are:

- DATA_WIDTH = 8
- PARITY_BITS = 4
- CODE_WIDTH = 12

For an 8-bit input data word, four parity bits are required because:

$$2^r \geq m + r + 1$$

For $m = 8$and $r = 4$:

$$2^4 \geq 8 + 4 + 1$$

$$16 \geq 13$$

Therefore, the resulting Hamming codeword contains:

$$n = m + r = 8 + 4 = 12$$

bits.

Code – globals.sv

```
`ifndef GLOBALS_SV
`define GLOBALS_SV
`timescale 1ns/1ps
parameter int DATA_WIDTH  = 8;
parameter int PARITY_BITS = 4;
parameter int CODE_WIDTH  = 12; // DATA_WIDTH + PARITY_BITS
`endif
```

2. Interface – interface.sv

The interface.sv file defines the SystemVerilog interface used for communication between the testbench components and the Hamming ECC design.

The interface groups all important signals into a single structure. This simplifies connectivity and allows the driver and monitor to access the DUT signals through a virtual interface.

The main signals are:

- tx_data – Data transmitted to the encoder
- encoded_codeword – Hamming codeword generated by the encoder
- corrupted_codeword – Codeword after single-bit error injection
- rx_data – Recovered data from the decoder
- error_flag – Indicates whether an error was detected
- error_pos – Indicates the detected error position

Code – interface.sv

```
`ifndef INTERFACE_SV
`define INTERFACE_SV
`include "globals.sv"
interface hamming_if;
   logic [DATA_WIDTH-1:0]  tx_data;
   logic [CODE_WIDTH-1:0]  encoded_codeword;
   logic [CODE_WIDTH-1:0]  corrupted_codeword;
```

```
    logic [DATA_WIDTH-1:0]  rx_data;
    logic  error_flag;
    logic [PARITY_BITS-1:0] error_pos;
endinterface
`endif
```

3. Transaction – transaction.sv

The transaction.sv file defines the transaction class used to represent one complete verification transaction.

The class contains the transmitted data, encoded codeword, corrupted codeword, recovered data, error status, and detected error position.

The tx_data variable is declared as rand, allowing the testbench to generate different input patterns automatically.

The display() function is used to print transaction information in the simulation console.

Code – transaction.sv

```
`ifndef TRANSACTION_SV
`define TRANSACTION_SV
`include "globals.sv"
class transaction;
    rand logic [DATA_WIDTH-1:0]  tx_data;
    logic      [CODE_WIDTH-1:0]  encoded_codeword;
    logic      [CODE_WIDTH-1:0]  corrupted_codeword;
    logic      [DATA_WIDTH-1:0]  rx_data;
    logic                        error_flag;
    logic      [PARITY_BITS-1:0] error_pos;
    function void display(string tag = "");
        $display("[%s] TX: 0x%h | Encoded: %b | Corrupted: %b | RX: 0x%h | Err: %b | Pos: %0d",
                 tag, tx_data, encoded_codeword, corrupted_codeword, rx_data, error_flag, error_pos);
    endfunction
endclass
`endif
```

4. Generator – generator.sv

The generator.sv file is responsible for generating randomized transactions.

The generator creates a new transaction object, randomizes the input data, and sends the transaction to the driver through the gen2drv mailbox.

The default number of transactions is 20. Therefore, the testbench can automatically verify the design using 20 different randomized input values.

The gen_done event indicates that all transactions have been generated.

Code – generator.sv

```
`ifndef GENERATOR_SV
`define GENERATOR_SV
`include "transaction.sv"
class generator;
   mailbox #(transaction) gen2drv;
   event gen_done;
   int count = 20;
   function new(mailbox #(transaction) gen2drv);
      this.gen2drv = gen2drv;
   endfunction
   task main();
      transaction tr;
      repeat (count) begin
         tr = new();
         if (!tr.randomize()) $error("[GEN] Randomization failed!");
         gen2drv.put(tr);
      end
      -> gen_done;
   endtask
endclass
`endif`
```

5. Driver – driver.sv

The driver.sv file receives transactions from the generator and drives the input data to the Hamming encoder.

After the encoder generates the codeword, the driver models a communication or storage channel where a single-bit error may occur.

The driver randomly decides whether an error should be introduced. If an error is selected, a random bit of the codeword is inverted.This provides automatic single-bit error injection during verification.

Code – driver.sv

```
`ifndef DRIVER_SV
`define DRIVER_SV
`include "transaction.sv"
class driver;
   virtual hamming_if vif;
   mailbox #(transaction) gen2drv;
   function new(virtual hamming_if vif, mailbox #(transaction) gen2drv);
      this.vif = vif;
      this.gen2drv = gen2drv;
   endfunction
   task main();
      transaction tr;
      int bit_to_flip;
      forever begin
         gen2drv.get(tr);
         // Drive data to Encoder
         vif.tx_data = tr.tx_data;
         #5;
         // Channel transmission with random single-bit error injection
         vif.corrupted_codeword = vif.encoded_codeword;
         if ($urandom_range(0, 1)) begin
            bit_to_flip = $urandom_range(0, CODE_WIDTH-1);
            vif.corrupted_codeword[bit_to_flip] = ~vif.corrupted_codeword[bit_to_flip];
         end
         #15;
      end
   endtask
endclass
`endif
```

6. Monitor – monitor.sv

The monitor.sv file observes the signals produced during simulation.

The monitor samples the transmitted data, encoded codeword, corrupted codeword, recovered data, error flag, and detected error position.

These values are stored in a transaction object and transferred to the scoreboard through the mon2scb mailbox.

Thus, the monitor provides the scoreboard with the actual output generated by the DUT.

Code – monitor.sv

```
`ifndef MONITOR_SV
`define MONITOR_SV
`include "transaction.sv"
class monitor;
  virtual hamming_if vif;
  mailbox #(transaction) mon2scb;
  function new(virtual hamming_if vif, mailbox #(transaction) mon2scb);
    this.vif = vif;
    this.mon2scb = mon2scb;
  endfunction
  task main();
    transaction tr;
    forever begin
      #18;
      tr = new();
      tr.tx_data          = vif.tx_data;
      tr.encoded_codeword  = vif.encoded_codeword;
      tr.corrupted_codeword = vif.corrupted_codeword;
      tr.rx_data          = vif.rx_data;
      tr.error_flag       = vif.error_flag;
      tr.error_pos        = vif.error_pos;
      mon2scb.put(tr);
    end
  endtask
```

```
endclass

`endif`
```

7. Scoreboard – scoreboard.sv

The scoreboard.sv file performs the main functional checking of the verification environment.The scoreboard receives the transaction collected by the monitor and compares the transmitted data with the recovered data.

The verification condition is:

$$\boxed{RX_DATA = TX_DATA}$$

If both values are equal, the transaction is classified as PASS. Otherwise, it is classified as FAIL.The scoreboard maintains separate counters for passed and failed transactions.

Code – scoreboard.sv

```
`ifndef SCOREBOARD_SV

`define SCOREBOARD_SV

`include "transaction.sv"

class scoreboard;

  mailbox #(transaction) mon2scb;

  int pass_count = 0;

  int fail_count = 0;

  function new(mailbox #(transaction) mon2scb);

    this.mon2scb = mon2scb;

  endfunction

  task main();

    transaction tr;

    forever begin

      mon2scb.get(tr);

      if (tr.rx_data == tr.tx_data) begin

        $display("[SCB PASS] TX: 0x%h | RX: 0x%h | Err Detected: %b | Pos Detected: %0d",

            tr.tx_data, tr.rx_data, tr.error_flag, tr.error_pos);

        pass_count++;

      end else begin

        $error("[SCB FAIL] MISMATCH! TX: 0x%h | RX: 0x%h",

          tr.tx_data, tr.rx_data);
```

```
            fail_count++;
          end
        end
    endtask
endclass
`endif`
```

8. Test Environment – pgtest.sv

The pgtest.sv file integrates all the verification components into a complete test environment.It creates the generator, driver, monitor, and scoreboard objects. Two mailboxes are used for communication between the components:

- gen2drv – Generator to Driver
- mon2scb – Monitor to Scoreboard

The components are executed concurrently using fork...join_any.

After all transactions are completed, the test displays the total PASS and FAIL count.

Code – pgtest.sv

```
`ifndef PGTEST_SV
`define PGTEST_SV
`include "generator.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"
class pgtest;
   generator  gen;
   driver     drv;
   monitor    mon;
   scoreboard scb;
   mailbox #(transaction) gen2drv;
   mailbox #(transaction) mon2scb;
   virtual hamming_if vif;
   function new(virtual hamming_if vif);
      this.vif = vif;
      gen2drv  = new();
```

```
        mon2scb  = new();
        gen = new(gen2drv);
        drv = new(vif, gen2drv);
        mon = new(vif, mon2scb);
        scb = new(mon2scb);
    endfunction
    task run();
  $display("\n============================================================");
        $display("    STARTING HAMMING CODE LAYERED TESTBENCH");
        $display("============================================================\n");
        fork
            gen.main();
            drv.main();
            mon.main();
            scb.main();
        join_any
        @gen.gen_done;
        #50;
  $display("\n============================================================");
        $display("    VERIFICATION COMPLETE | PASSED: %0d | FAILED: %0d",
                scb.pass_count, scb.fail_count);
        $display("============================================================\n");
        $finish;
    endtask
endclass
`endif`
```

9. Top-Level Testbench – hamming_tb.sv

The hamming_tb.sv file is the top-level module for the layered verification environment.It instantiates the Hamming interface, encoder DUT, decoder DUT, and pgtest test environment.

The encoder receives the transmitted data and generates the Hamming codeword. The decoder receives the potentially corrupted codeword and attempts to recover the original data.

The testbench also enables waveform dumping using $dumpfile and $dumpvars.

Code – hamming_tb.sv

```
`timescale 1ns/1ps
`include "interface.sv"
`include "hamming_encoder.sv"
`include "pgtest.sv"
module hamming_tb;
    // Interface
    hamming_if vif();
    // Auto-dump all signals to waveform
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, hamming_tb);
    end
    // DUT 1: Encoder
    hamming_encoder #(
        .DATA_WIDTH(DATA_WIDTH)
    ) enc_dut (
        .data_in(vif.tx_data),
        .codeword(vif.encoded_codeword)
    );
    // DUT 2: Decoder
    hamming_decoder #(
        .DATA_WIDTH(DATA_WIDTH)
    ) dec_dut (
        .codeword_in(vif.corrupted_codeword),
        .data_out(vif.rx_data),
        .error_detected(vif.error_flag),
        .error_pos(vif.error_pos)
    );
    // Test environment
    pgtest test;
    initial begin
```

```
    test = new(vif);
    test.run();
  end
endmodule
```

10. Directed Testbench – tb_hamming_encoder.sv

The tb_hamming_encoder.sv file provides directed testing of the Hamming encoder and decoder.Unlike the randomized layered testbench, this testbench uses predefined data values and specific error positions.

The testbench performs thre main tests:

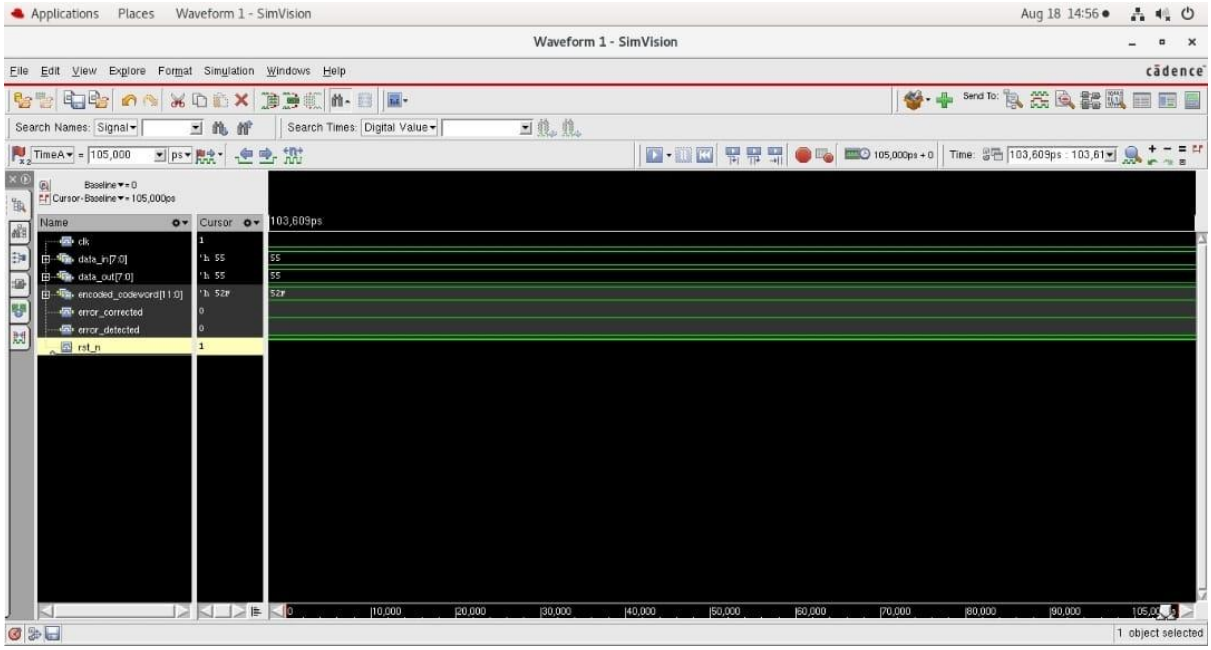
1. No error condition.
2. Single-bit error at position 3.
3. Single-bit error at position 7.

This directed testing provides a simple way to demonstrate the basic error correction functionality of the design.

Code – tb_hamming_encoder.sv

```
`timescale 1ns/1ps
`include "globals.sv"
module tb_hamming_encoder;
  logic [DATA_WIDTH-1:0]  data_in;
  logic [CODE_WIDTH-1:0]  codeword;
  logic [CODE_WIDTH-1:0]  noisy_codeword;
  logic [DATA_WIDTH-1:0]  data_out;
  logic                   error_detected;
  logic [PARITY_BITS-1:0] error_pos;
  hamming_encoder #(
    .DATA_WIDTH(DATA_WIDTH)
  ) enc (
    .data_in(data_in),
    .codeword(codeword)
  );
  hamming_decoder #(
    .DATA_WIDTH(DATA_WIDTH)
  ) dec (
```

```
        .codeword_in(noisy_codeword),
        .data_out(data_out),
        .error_detected(error_detected),
        .error_pos(error_pos)
    );
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_hamming_encoder);
        // Test 1: No Error
        data_in = 8'hA5;        #10;
        noisy_codeword = codeword;        #10;
        $display("[DIRECTED TB] No Error -> TX: 0x%h | RX: 0x%h | Err: %b",
                data_in, data_out, error_detected);
        // Test 2: Injected bit flip at position 3
        noisy_codeword[2] = ~noisy_codeword[2];   #10;
        $display("[DIRECTED TB] Bit 3 Flip -> TX: 0x%h | RX: 0x%h | Err: %b | Pos: %0d",
                data_in, data_out, error_detected, error_pos);
        // Test 3: New data with bit flip at position 7
        data_in = 8'h3C;          #10;
        noisy_codeword = codeword;
        noisy_codeword[6] = ~noisy_codeword[6];        #10;
        $display("[DIRECTED TB] Bit 7 Flip -> TX: 0x%h | RX: 0x%h | Err: %b | Pos: %0d",
                data_in, data_out, error_detected, error_pos);        #20;
  $finish;
    end
endmodule
```



COVERAGE ANALYSIS

Coverage analysis is performed to evaluate the completeness of the verification process and to ensure that all functional scenarios of the proposed Parameterizable Hamming Code Single Error Correction (SEC) ECC System have been exercised during simulation. Functional coverage provides confidence that the RTL design has been thoroughly verified before proceeding to further implementation stages.

In this project, coverage analysis is carried out using the Cadence Xcelium Simulator during RTL simulation. The coverage analysis focuses on the encoding, error injection, syndrome generation, error detection, error correction, and data recovery operations.

The coverage analysis confirms that the proposed Hamming SEC ECC system correctly handles:

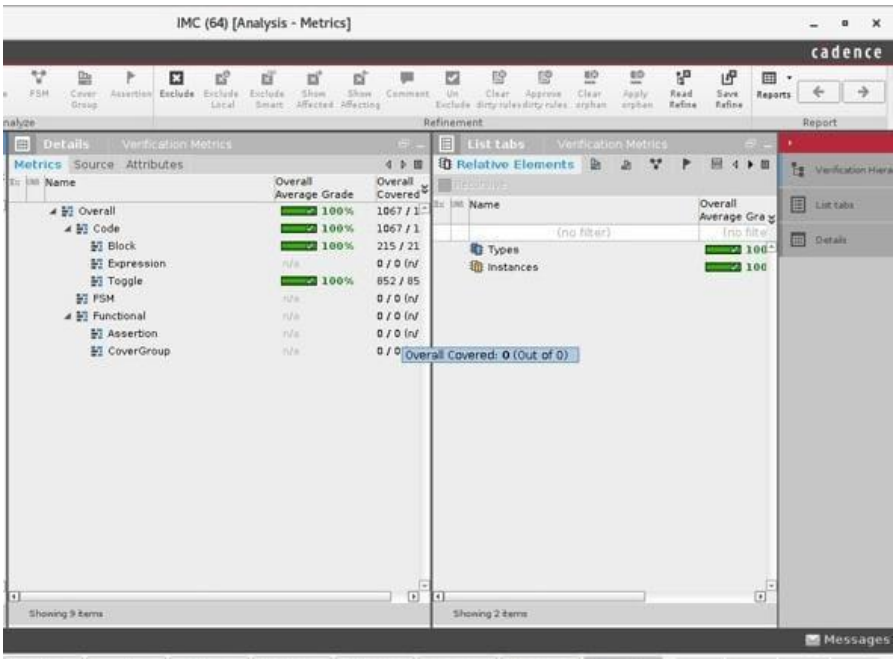
- Different input data patterns.
- Hamming codeword generation.
- Correct placement of parity bits.
- Correct calculation of even parity bits.
- No-error transmission condition.
- Single-bit error injection.
- Single-bit errors at different codeword positions.
- Correct syndrome generation.
- Correct identification of the erroneous bit position.
- Correct inversion of the erroneous bit.
- Correct recovery of the original data.

- Detection of error-free codewords using zero syndrome.
- Verification of encoder and decoder operation.

COVERAGE ANALYSIS:

The generated coverage report confirms that the major functional paths of the proposed Hamming SEC ECC system are exercised during simulation. The verification process validates Hamming encoding, parity generation, error injection, syndrome calculation, error detection, error correction, and recovered data verification.

The 100% functional coverage demonstrates that the verification environment has comprehensively exercised the planned operating conditions of the parameterizable Hamming Code SEC ECC system.



CONSTRAINTS FILE CODE :

SDC FILE CODE

```
create_clock -name clk -period 1.0 -waveform {0 0.5} [get_ports clk]

set_clock_transition -rise 0.05 [get_clocks clk]

set_clock_transition -fall 0.05 [get_clocks clk]

set_clock_uncertainty 0.05 [get_clocks clk]

set_input_delay -max 0.20 -clock [get_clocks clk] [get_ports tx_data]

set_output_delay -max 0.20 -clock [get_clocks clk] [get_ports rx_data]

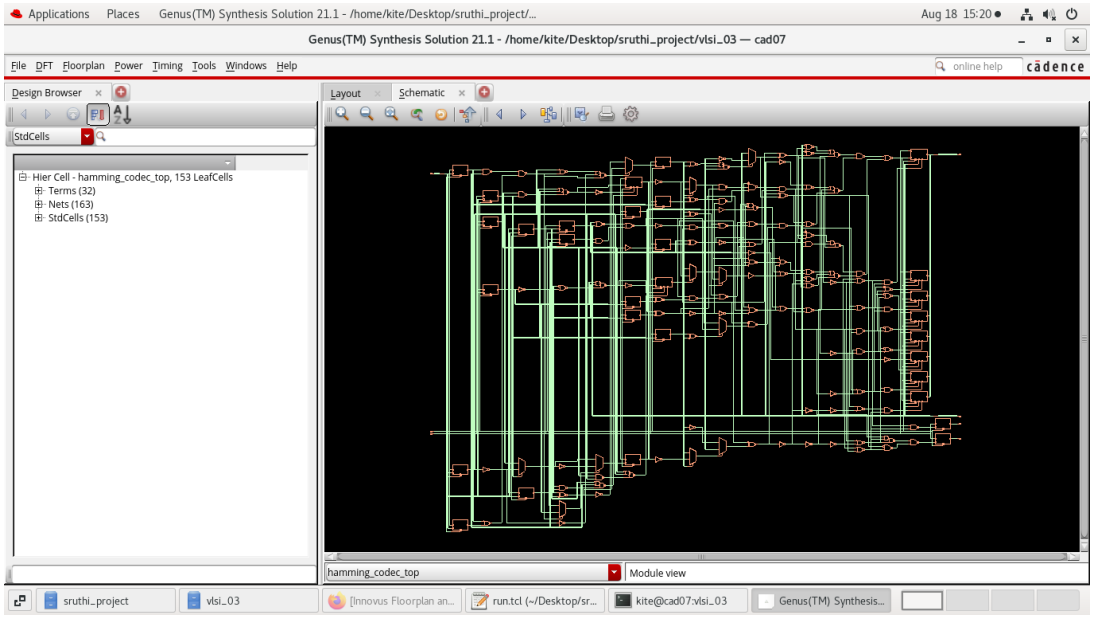
set_input_delay -max 0.20 -clock [get_clocks clk] [get_ports corrupted_codeword]

set_output_delay -max 0.20 -clock [get_clocks clk] [get_ports encoded_codeword]

set_output_delay -max 0.20 -clock [get_clocks clk] [get_ports error_flag]
```

set_output_delay -max 0.20 -clock [get_clocks clk] [get_ports error_pos]

Simulation Command: genus -f run.tcl



Run Tcl File:

A Tcl (Tool Command Language) script is used to automate the logic synthesis process in Cadence Genus. Instead of executing synthesis commands manually, all required synthesis commands are written in a single Tcl script, allowing the complete synthesis flow to be executed efficiently and consistently.

The Tcl script performs the following operations:

- Sets the technology library search path.
- Loads the standard cell library.
- Reads the SystemVerilog RTL design.
- Elaborates the RTL design.
- Reads the timing constraint file (.sdc).
- Configures synthesis options.
- Performs generic synthesis, technology mapping, and optimization.
- Generates the synthesized gate-level netlist.
- Writes the synthesized constraint file.
- Displays the synthesized design in the GUI.
- Generates timing, power, area, and gate reports.

The Tcl script used for the proposed Parameterizable Hamming Code Single Error Correction (SEC) ECC System is given below:

```
set_db init_lib_search_path /home/install/FOUNDRY/digital/45nm/dig/lib
set_db library slow.lib
read_hdl {./hamming_encoder.sv ./hamming_decoder.sv ./hamming_top.sv}
elaborate
read_sdc ./constraints_hamming.sdc
set_db use_scan_seqs_for_non_dft false
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
syn_generic
syn_map
syn_opt
write_hdl > hamming_ecc_netlist.v
write_sdc > hamming_ecc_tool.sdc
gui_show
report timing > hamming_ecc_timing.rpt
report power > hamming_ecc_power.rpt
report area > hamming_ecc_area.rpt
report gates > hamming_ecc_gates.rpt
```

Gate-Level Netlist

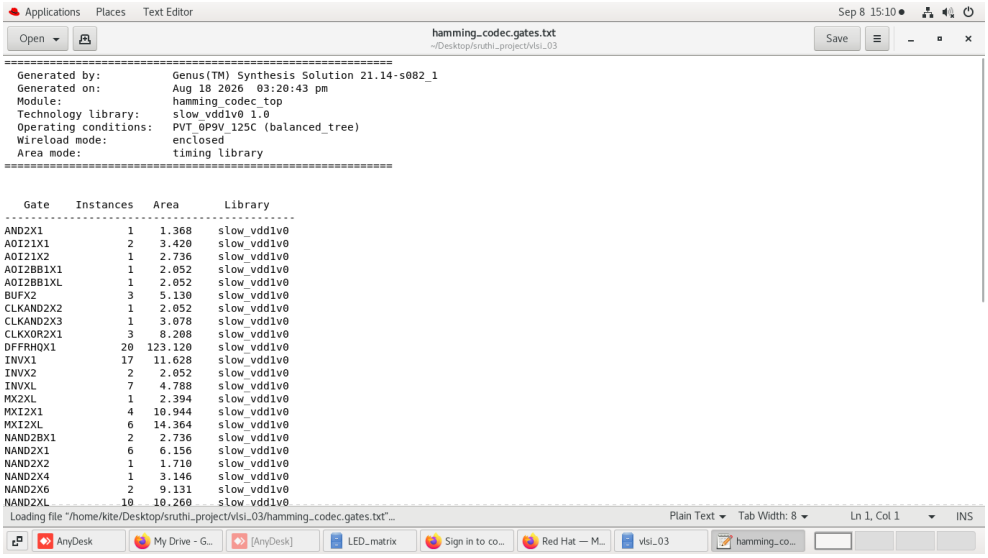
After successful logic synthesis, Cadence Genus generates the gate-level netlist for the proposed Parameterizable Hamming Code Single Error Correction (SEC) ECC System. The synthesized netlist consists of technology-specific standard cells from the target CMOS library.

The gate-level implementation preserves the functionality of the original RTL design while optimizing the circuit for area, timing, and power. The generated netlist represents the technology-mapped implementation of the Hamming encoder and decoder and can be used as input for subsequent physical design stages, including floorplanning, placement, clock tree synthesis, routing, and timing analysis.

The corresponding synthesized timing constraint file is generated as:

hamming_codec_tool.sdc

The generated timing, power, area, and gate reports are used to evaluate the implementation characteristics of the proposed Hamming SEC ECC system.



```
Applications  Places  Text Editor                                   Sep 8 15:10
Open   hamming_codec.gates.txt  ~/Desktop/sruthi_project/vlsi_03   Save

============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Aug 18 2026  03:20:43 pm
  Module:                 hamming_codec_top
  Technology library:     slow_vdd1v0 1.0
  Operating conditions:   PVT_0P9V_125C (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

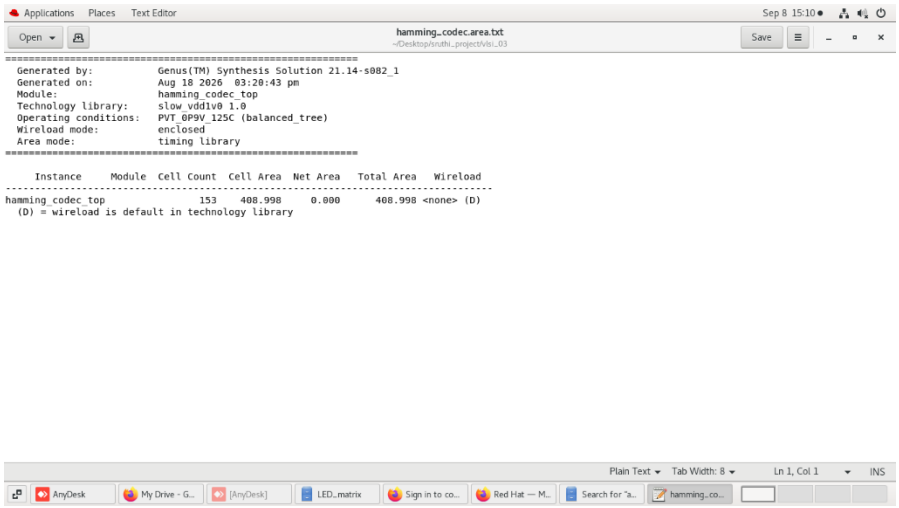

   Gate    Instances   Area       Library
-----------------------------------------------
AND2X1             1    1.368    slow_vdd1v0
AOI21X1            2    3.420    slow_vdd1v0
AOI21X2            1    2.736    slow_vdd1v0
AOI2BB1X1          1    2.052    slow_vdd1v0
AOI2BB1XL          1    2.052    slow_vdd1v0
BUFX2              3    5.130    slow_vdd1v0
CLKAND2X2          1    2.052    slow_vdd1v0
CLKAND2X3          1    3.078    slow_vdd1v0
CLKXOR2X1          3    8.208    slow_vdd1v0
DFFRHQX1          20  123.120    slow_vdd1v0
INVX1             17   11.628    slow_vdd1v0
INVX2              2    2.052    slow_vdd1v0
INVXL              7    4.788    slow_vdd1v0
MX2XL              1    2.394    slow_vdd1v0
MXI2X1             4   10.944    slow_vdd1v0
MXI2XL             6   14.364    slow_vdd1v0
NAND2BX1           2    2.736    slow_vdd1v0
NAND2X1            6    6.156    slow_vdd1v0
NAND2X2            1    1.710    slow_vdd1v0
NAND2X4            1    3.146    slow_vdd1v0
NAND2X6            2    9.131    slow_vdd1v0
NAND2XL           10   10.260    slow_vdd1v0
Loading file "/home/kite/Desktop/sruthi_project/vlsi_03/hamming_codec.gates.txt"...   Plain Text  Tab Width: 8  Ln 1, Col 1  INS
```

Synthesis Reports

Cadence Genus automatically generates several reports after synthesis to evaluate the quality and performance of the synthesized design. These reports provide information about the design area, timing performance, power consumption, and gate utilization of the proposed Parameterizable Hamming Code Single Error Correction (SEC) ECC System.

Area Report

The Area Report provides information about the hardware resources utilized by the synthesized Parameterizable Hamming Code SEC ECC System. It includes the total standard cell area, the number of logic cells used, combinational logic, and the overall resource utilization of the design.The area report helps evaluate the hardware complexity and silicon area required for ASIC implementation. It also provides an indication of the efficiency of the synthesized Hamming encoder and decoder and can be used to compare different design configurations or parameter values.



```
Applications  Places  Text Editor                                   Sep 8 15:10
Open   hamming_codec.area.txt  ~/Desktop/sruthi_project/vlsi_03   Save

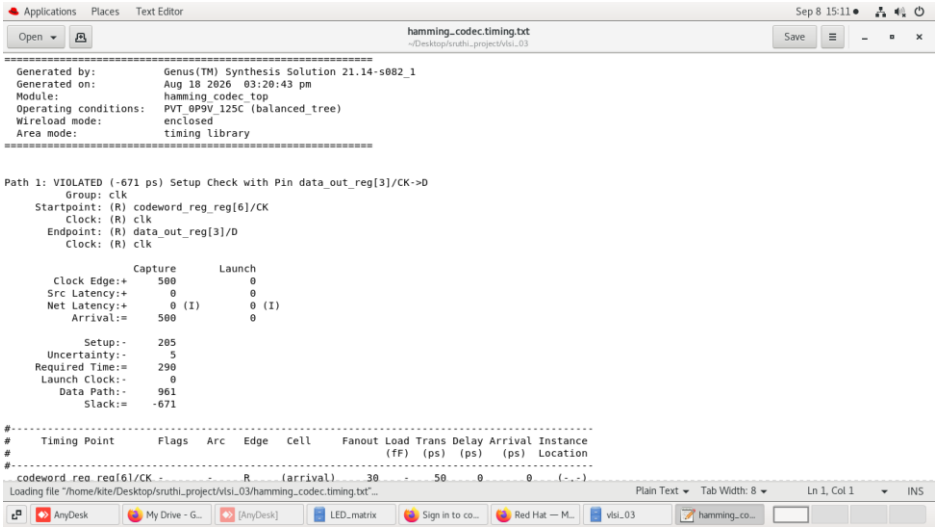
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Aug 18 2026  03:20:43 pm
  Module:                 hamming_codec_top
  Technology library:     slow_vdd1v0 1.0
  Operating conditions:   PVT_0P9V_125C (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

     Instance     Module  Cell Count  Cell Area  Net Area   Total Area   Wireload
-----------------------------------------------------------------------------------
hamming_codec_top                153    408.998     0.000      408.998 <none> (D)
  (D) = wireload is default in technology library
Plain Text  Tab Width: 8  Ln 1, Col 1  INS
```

Timing Report

The Timing Report verifies that the synthesized Parameterizable Hamming Code Single Error Correction (SEC) ECC System satisfies all specified timing constraints. It analyzes the critical

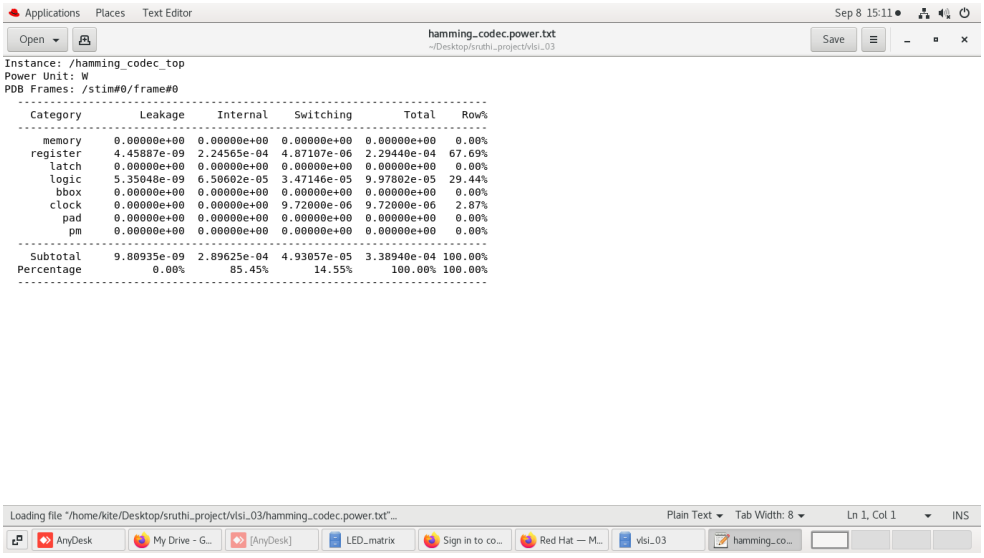
timing paths between the Hamming encoder, decoder, syndrome generation, and error correction logic, and ensures that the design meets the required operating frequency without setup or hold time violations.The timing report includes the following parameters:



Power Report

The Power Report provides an estimation of the total power consumed by the synthesized Parameterizable Hamming Code Single Error Correction (SEC) ECC System. It helps evaluate the energy efficiency of the design before physical implementation.

The report provides information about the power consumed by the Hamming encoder, decoder, and other synthesized logic components. It can be used to analyze the power requirements of the design and evaluate its suitability for low-power ASIC applications.



LOGIC EQUIVALENCE CHECKING (LEC):

Logic Equivalence Checking (LEC) is a crucial verification stage in the ASIC RTL-to-GDSII design flow. It verifies that the synthesized gate-level netlist generated by Cadence Genus is functionally identical to the original Register Transfer Level (RTL) design. This ensures that the logic synthesis process has not introduced any functional changes into the design.

In this project, Cadence Conformal LEC is used to compare the RTL implementation of the Parameterizable Hamming Code Single Error Correction (SEC) ECC System with the synthesized gate-level netlist. After successful verification, the synthesized design is confirmed to be functionally equivalent to the RTL design and is considered ready for the subsequent physical design stages.

LEC Environment

Parameter	**Description**
Tool	Cadence Conformal LEC
Golden Design	SystemVerilog RTL – Hamming ECC Design
Revised Design	Synthesized Gate-Level Netlist (hamming_ecc_netlist.v)
Library	90 nm Standard Cell Library (slow.lib)
Output	Equivalence Verification Report

LEC Implementation

The RTL design and synthesized gate-level netlist are imported into the Cadence Conformal LEC environment. The required technology library is loaded to interpret the standard cells used in the synthesized design. The RTL design is treated as the **Golden Design**, while the synthesized gate-level netlist is treated as the **Revised Design**.

After loading both designs, Conformal automatically maps the primary inputs, outputs, registers, and internal compare points. The equivalence checking process is then executed using Tcl commands. Once the comparison is completed, the tool generates a verification report indicating whether the synthesized netlist is functionally equivalent to the original RTL design.

LEC Setup Mode:

In the Setup Mode, the required technology library, RTL design, and synthesized gate-level netlist are loaded into the Conformal LEC environment using the following commands.

Simulation Command:

Read Technology Library

read library -liberty /home/install/FOUNDRY/digital/90nm/dig/lib/slow.lib

Read Golden RTL Design

read design -verilog hamming_top.v -golden

Read Revised Synthesized Netlist

read design -verilog hamming_ecc_netlist.v -revised

The above commands perform the following functions:

- read library loads the 90 nm CMOS standard cell library required for gate-level interpretation.

- read design -golden imports the original SystemVerilog RTL design as the Golden Design.
- read design -revised imports the synthesized gate-level netlist as the Revised Design.

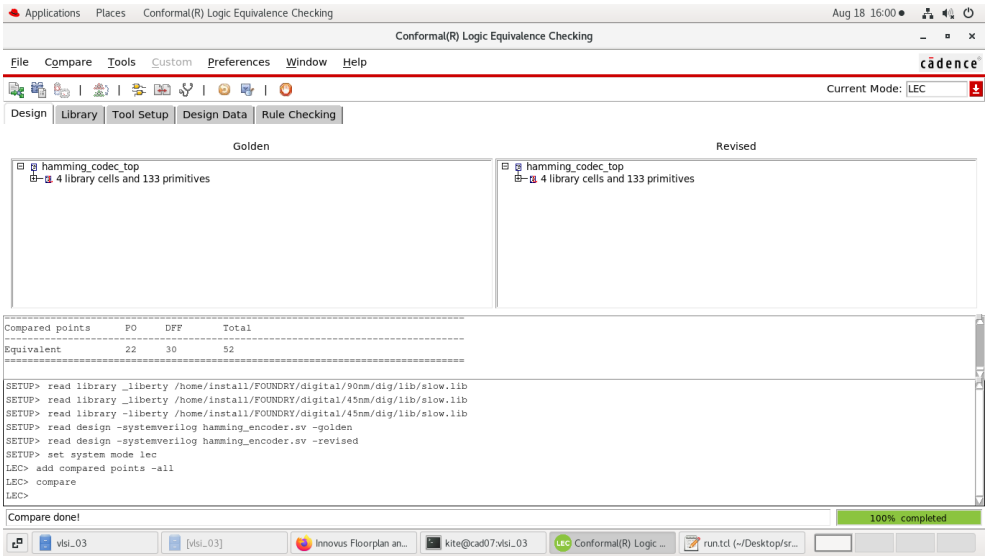
LEC Mode

After the setup process is completed, the tool enters LEC Mode, where the Golden RTL design and the Revised synthesized netlist are compared.

During this stage, Conformal automatically performs the following operations:

- Matches primary inputs and outputs.
- Maps registers and combinational logic.
- Identifies equivalent compare points.
- Detects unmatched or non-equivalent logic.
- Generates the equivalence verification report.

The comparison verifies that the synthesized gate-level netlist preserves the intended functionality of the original Hamming SEC ECC RTL implementation.



Mapping Manager

The Mapping Manager displays the correspondence between the Golden RTL design and the Revised synthesized netlist.

It provides information such as:

- Number of mapped compare points.
- Number of unmapped compare points.
- Primary input mapping.
- Primary output mapping.

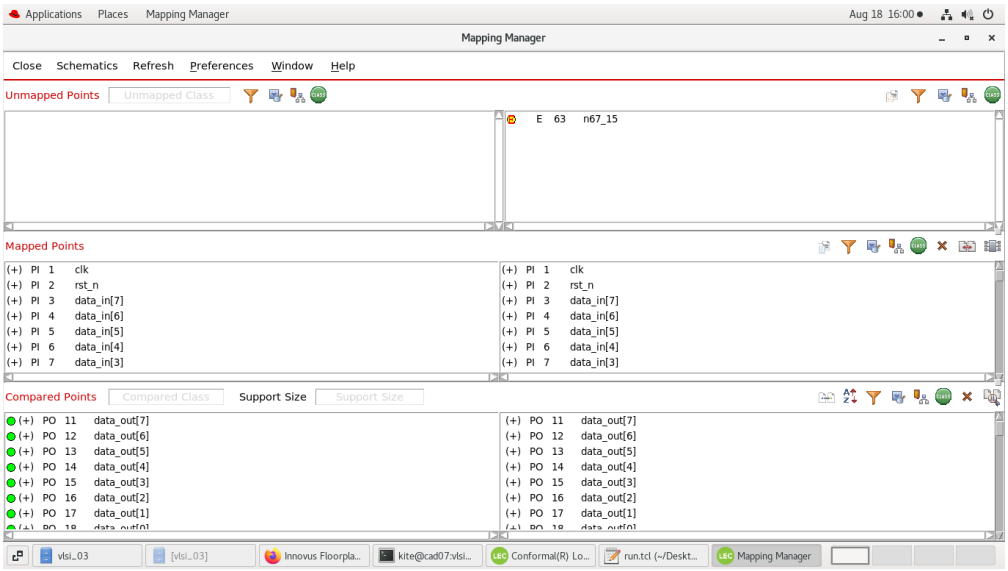
- Register mapping.
- Compare point summary.

The Mapping Manager is used to verify that the logical points in the RTL design are correctly matched with their corresponding points in the synthesized gate-level netlist.

Verification Result

After completing the equivalence checking process, Cadence Conformal reports whether the Golden RTL Design and the Revised Gate-Level Netlist are Equivalent or Not Equivalent.

A successful LEC result confirms that the synthesis process has preserved the intended functionality of the Parameterizable Hamming Code SEC ECC System. Therefore, the synthesized netlist can proceed to the subsequent physical design stages, including floorplanning, placement, clock tree synthesis, routing, static timing analysis, and GDSII generation.



PHYSICAL DESIGN

Physical design is the process of converting the synthesized gate-level netlist into a physical integrated circuit layout. It transforms the logical design into a geometric representation by placing standard cells, creating the power distribution network, establishing clock distribution, routing interconnections, and verifying the design for timing, power, and physical constraints.

For the proposed Parameterizable Hamming Code Single Error Correction (SEC) ECC System, the physical implementation is carried out using the Cadence Innovus Implementation System. The synthesized netlist generated from Cadence Genus serves as the input for the physical design flow. During this stage, the design is optimized to satisfy area, timing, power, and routing requirements while maintaining compatibility with the 90 nm CMOS standard cell technology.

The physical design process follows a sequence of stages including floorplanning, power planning, placement, clock tree synthesis (CTS), routing, static timing analysis (STA), power analysis, and GDSII generation. Each stage is verified and optimized before proceeding to the next stage.

Command Description

Command	Description
csh	Starts the C Shell environment.
source /home/install/cshrc	Loads the Cadence environment variables, technology libraries, and tool settings.
innovus	Launches the Cadence Innovus Physical Design Tool.

FLOORPLANNING:

Floorplanning is the first stage of the physical design process in ASIC implementation. It defines the physical structure of the integrated circuit by determining the die size, core area, placement boundaries, standard cell rows, I/O pin locations, and power distribution network.

For the proposed Parameterizable Hamming Code SEC ECC System, the synthesized gate-level netlist generated by Cadence Genus is imported into Cadence Innovus for floorplanning. During this stage, the core utilization, aspect ratio, and chip dimensions are specified according to the design requirements. Standard cell rows are created within the core region to accommodate the placement of logic cells.

Power planning is also considered during floorplanning by defining the VDD and VSS power distribution structures. The floorplan is designed to minimize routing congestion, reduce interconnect length, and improve the overall timing and performance of the Hamming ECC system.

After completing the floorplanning stage, the layout is verified to ensure that the core area, placement boundaries, standard cell rows, power structures, and I/O locations are correctly established.

POWER PLANNING:

Power planning is an important stage of the physical design process. Its primary objective is to establish a reliable power distribution network (PDN) that supplies stable VDD (Power) and VSS (Ground) connections to the standard cells in the design.

For the proposed Parameterizable Hamming Code SEC ECC System, power planning is performed using Cadence Innovus after floorplanning. Power rings are created around the core area, and power stripes are inserted across the design to distribute power uniformly to the standard cells.

Proper power planning helps minimize IR drop, electromigration, and voltage fluctuations. It also improves signal integrity and ensures reliable operation of the synthesized Hamming encoder and decoder during physical implementation.

PLACEMENT:

Placement is the process of positioning all standard cells within the predefined core area after floorplanning and power planning. The objective is to arrange the cells efficiently while satisfying timing, area, congestion, and power constraints.

For the proposed Parameterizable Hamming Code SEC ECC System, placement is performed using Cadence Innovus. GlobalNet Connection is established to connect the VDD and VSS nets throughout the design. The standard cells are then placed within the available core area.

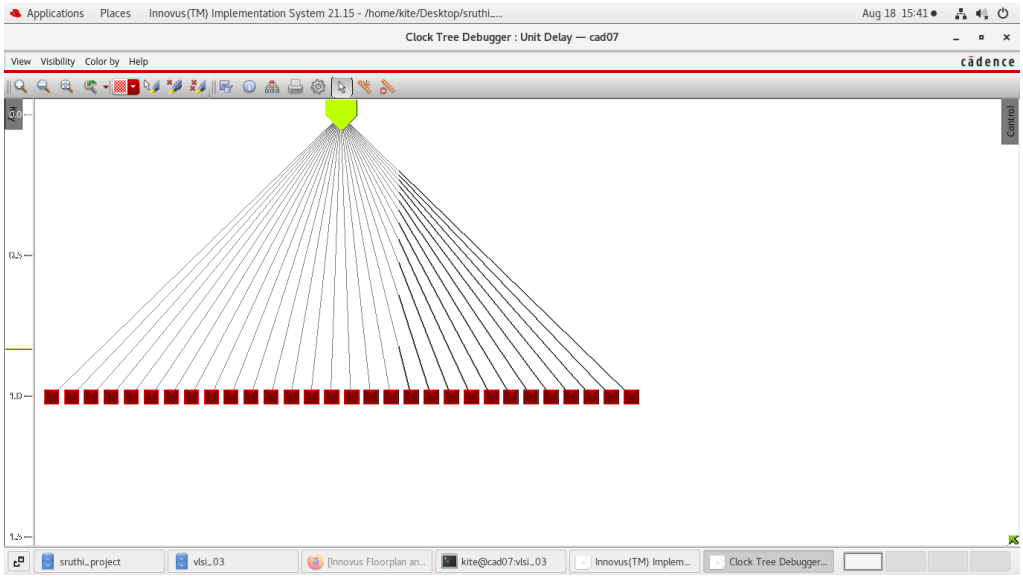
The placement engine optimizes cell locations to reduce interconnect length, minimize routing congestion, and improve timing performance. Global placement provides an initial optimized arrangement, followed by detailed placement to remove overlaps and achieve legal placement.After placement, the design is verified to ensure that all standard cells are legally positioned and properly connected to the power network. The optimized placement provides the foundation for the Clock Tree Synthesis stage.

Placement Operations

- **GlobalNet Connect:** Connects the global VDD and VSS nets to the standard cells.
- **Add Ring:** Creates power and ground rings around the core.
- **Add Stripe:** Inserts power stripes to distribute power throughout the design.
- **Standard Cell Placement:** Places the standard cells while optimizing timing, area, and routing congestion.

Clock Tree Synthesis (CTS)

Clock Tree Synthesis (CTS) is an important stage in the physical design flow for designs containing sequential logic. It creates an optimized clock distribution network for the sequential elements of the design.For the proposed Parameterizable Hamming Code SEC ECC System, CTS is performed using Cadence Innovus when a clocked RTL implementation is used. During CTS, clock buffers and inverters may be inserted to control clock skew, latency, and insertion delay.The CTS stage ensures that the clock reaches the required sequential elements reliably while satisfying the specified timing constraints. After successful CTS, the design proceeds to the routing stage.



ROUTING:

Routing is the process of establishing electrical connections between the placed standard cells according to the synthesized gate-level netlist. It creates the complete interconnection network required for the operation of the integrated circuit.

For the proposed Parameterizable Hamming Code SEC ECC System, routing is performed using Cadence Innovus after placement and CTS, where applicable. The tool connects the Hamming encoder, syndrome generation, error correction, decoder, input/output ports, clock network, and power connections using the available metal layers of the 90 nm CMOS technology.

The routing process is optimized to minimize wire length, signal delay, congestion, and crosstalk while satisfying the technology design rules.

Static Timing Analysis (STA)

Static Timing Analysis (STA) is a crucial verification stage in the ASIC physical design flow. It is used to verify whether the implemented design satisfies the specified timing constraints without applying functional input vectors.

For the proposed Parameterizable Hamming Code SEC ECC System, STA is performed after routing using the available Cadence timing-analysis tools. The timing engine analyzes the implemented signal paths and evaluates important timing parameters such as arrival time, required time, setup slack, hold slack, clock latency, and critical path delay.

Based on the timing results, the design can be optimized to eliminate timing violations and improve overall performance. Successful timing verification confirms that the implemented design satisfies the required timing constraints.

Timing Analysis Result

The timing analysis is used to determine whether the implemented Hamming SEC ECC System satisfies the specified timing requirements. Setup and hold checks are analyzed, and critical timing paths are identified for optimization when necessary.

Solving Setup and Hold Failing Paths

After routing, the implemented design is analyzed to identify setup and hold timing violations. The analysis uses the routed design database, extracted parasitic information, and timing constraints specified in the SDC file.

Timing analysis identifies the start point, end point, data path delay, clock path delay, and slack values of critical paths. Based on these results, appropriate timing optimization techniques can be applied.

Setup Failing Paths

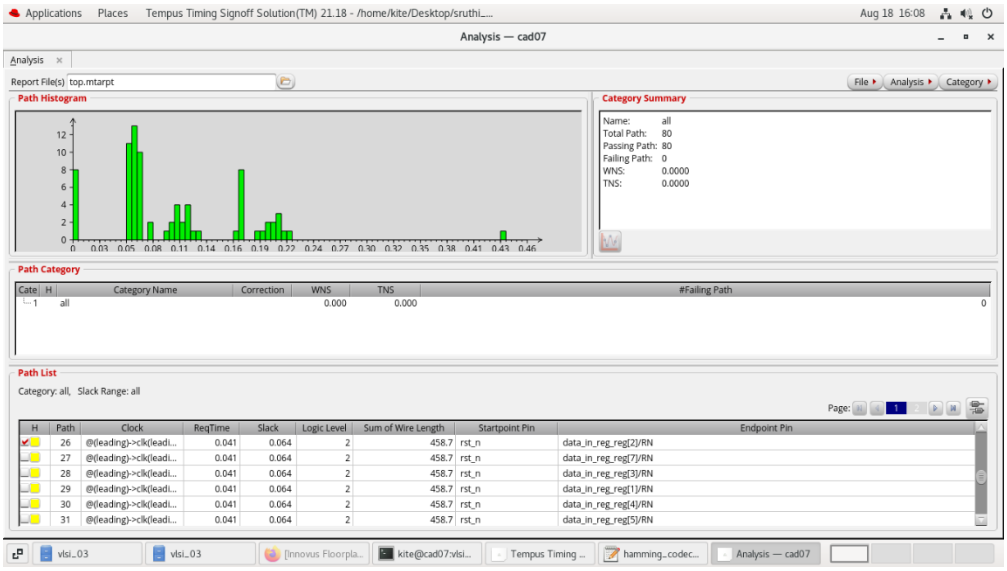
A setup violation occurs when data reaches the destination sequential element later than the required setup time before the active clock edge. This produces negative setup slack.

Setup violations can be reduced using techniques such as cell resizing, logic optimization, buffer insertion, interconnect optimization, and timing-driven placement or routing. The timing paths are re-analyzed after optimization to verify improvement in setup slack.

Hold Failing Paths

A hold violation occurs when data reaches the destination sequential element earlier than the required hold time after the active clock edge. This produces negative hold slack.

Hold violations can be corrected by increasing the delay of the affected data paths using appropriate buffering and optimization techniques. The design is then re-analyzed to ensure that the hold requirements are satisfied.



Timing Optimization and Verification

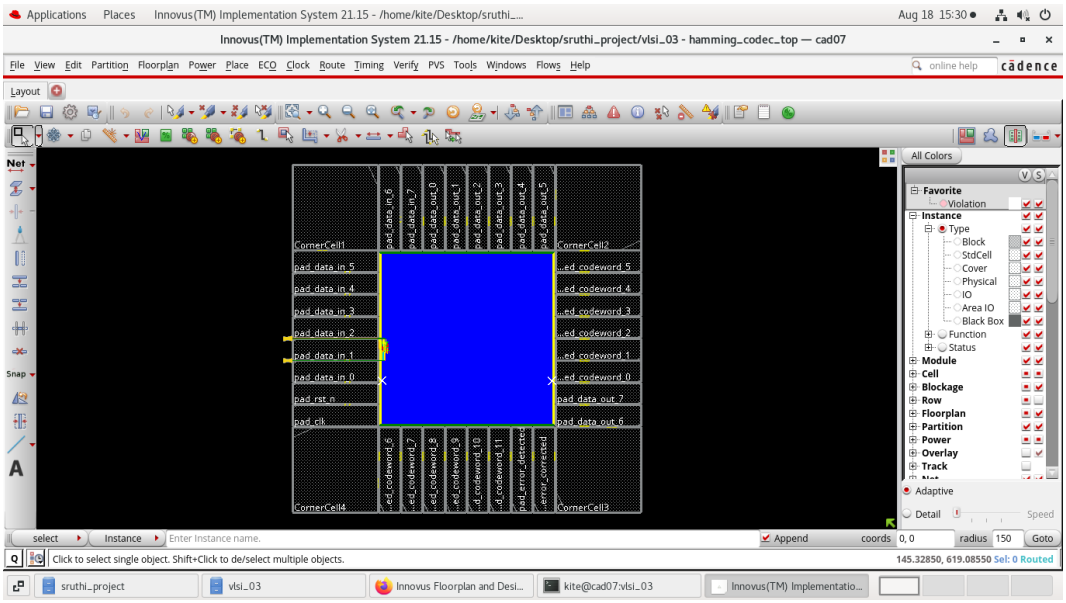
Timing closure is achieved through repeated timing analysis and optimization. The timing reports provide information about critical paths, slack values, clock characteristics, and propagation delays.

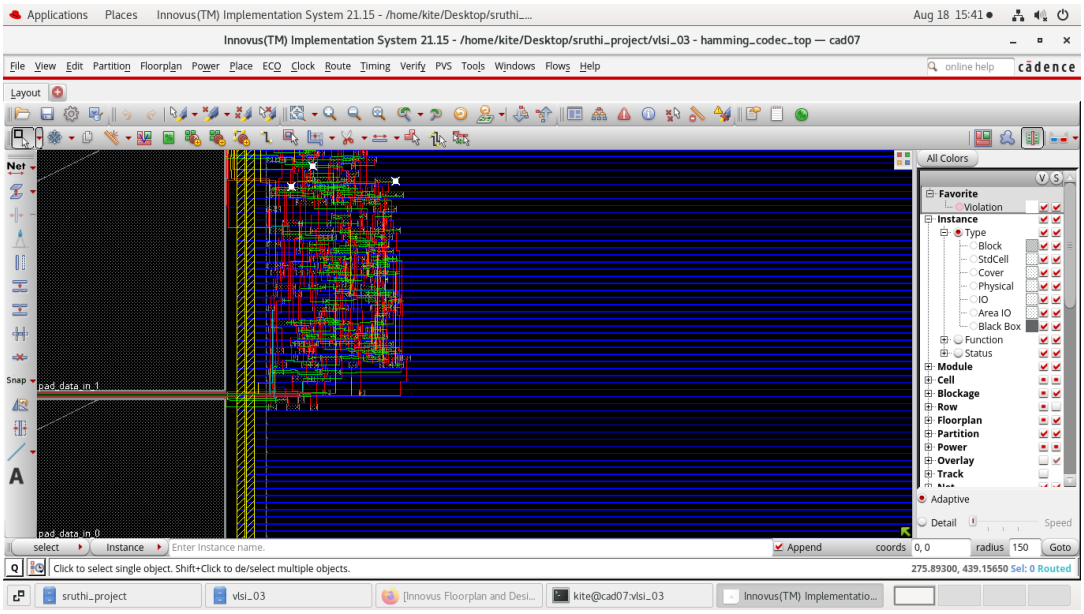
The analysis is performed under the specified Process, Voltage, and Temperature (PVT) conditions using the 90 nm CMOS standard cell library. This helps verify the robustness of the physical implementation under the required operating conditions.After timing optimization, the design is re-analyzed to confirm that the specified setup and hold timing requirements are satisfied.

Power Analysis

Power analysis is an essential stage in the ASIC physical design flow that estimates the power consumed by the implemented design. It evaluates the contributions from internal power, switching power, and leakage power.

For the proposed **Parameterizable Hamming Code SEC ECC System**, power analysis is performed after physical implementation and timing verification. The routed design, technology library, and switching activity information are used to estimate the overall power consumption.The generated power report provides information about the power consumed by different parts of the Hamming ECC design and helps evaluate its energy efficiency.





GDSII GENERATION

GDSII (Graphic Data System II) Generation is the final stage of the ASIC physical design flow. It involves exporting the completed physical layout into a GDSII file, which is the standard format used to represent the physical geometry of an integrated circuit.

For the proposed **Parameterizable Hamming Code SEC ECC System**, the GDSII file is generated using Cadence Innovus after completing the required physical implementation and verification stages, including floorplanning, power planning, placement, CTS where applicable, routing, timing verification, and power analysis.

The generated GDSII layout contains the physical representation of the standard cells, metal interconnections, vias, power network, clock network, and other technology-specific layout information. The layout can subsequently be subjected to physical verification processes such as Design Rule Checking (DRC), Layout Versus Schematic (LVS), and Parasitic Extraction (PEX).

Successful GDSII generation indicates completion of the physical implementation flow for the proposed Hamming SEC ECC System.



RESULTS AND DISCUSSION:

The proposed **Parameterizable Hamming Code Single Error Correction (SEC) ECC System** was designed and verified using SystemVerilog. The functional behavior of the Hamming encoder and decoder was verified using a dedicated testbench with randomized data and single-bit error injection.The RTL design generates Hamming codewords by inserting parity bits at the required positions. At the receiver, the decoder calculates the syndrome to identify the location of a single-bit error. The detected erroneous bit is corrected, and the original data is recovered.The RTL design can be synthesized using Cadence Genus, where the SystemVerilog implementation is mapped to the 90 nm CMOS standard cell library. The synthesis process generates a technology-mapped gate-level netlist while preserving the intended functionality of the RTL design.Logic Equivalence Checking using Cadence Conformal can then be used to verify that the synthesized gate-level netlist maintains functional equivalence with the original RTL design.

The physical implementation can be carried out using Cadence Innovus through floorplanning, power planning, placement, clock tree synthesis where applicable, routing, timing analysis, power analysis, and GDSII generation.Overall, the proposed Hamming SEC ECC system provides a parameterizable architecture capable of detecting and correcting single-bit errors and recovering the original data. The design methodology demonstrates the application of RTL design, functional verification, synthesis, equivalence checking, and ASIC physical implementation techniques.

CHALLENGES FACED

During the implementation of the proposed **Parameterizable Hamming Code SEC ECC System**, several technical challenges were encountered at different stages of the design flow. These challenges include:

- Understanding the complete RTL-to-GDSII ASIC design flow and the functionality of different Cadence EDA tools.
- Calculating the required number of parity bits for different input data widths.

- Correctly placing parity bits at positions 1, 2, 4, 8, 16, and so on.
- Implementing even-parity generation for the Hamming encoder.
- Developing syndrome-generation logic for identifying single-bit error positions.
- Correcting the detected erroneous bit without affecting the remaining codeword.
- Designing a parameterizable RTL architecture for different data widths.
- Developing a verification environment capable of injecting single-bit errors at different codeword positions.
- Debugging functional errors identified during RTL simulation.
- Achieving successful synthesis while preserving the functionality of the ECC system.
- Performing physical implementation while satisfying area, timing, power, and routing requirements.

CONCLUSION:

The proposed **Parameterizable Hamming Code Single Error Correction (SEC) ECC System** was successfully designed using SystemVerilog with the objective of detecting and correcting single-bit errors in digital data. The system uses Hamming coding to insert parity bits into the input data and generate an encoded codeword.At the receiver, parity checks are performed to calculate the syndrome. A zero syndrome indicates an error-free codeword, while a non-zero syndrome identifies the position of a single-bit error. The corresponding erroneous bit can then be inverted to recover the correct codeword, followed by extraction of the original data.

The parameterizable architecture provides flexibility to support different input data widths by automatically determining the required number of parity bits. Functional verification using randomized data and single-bit error injection provides a systematic method for validating the encoder and decoder operations.The design can be further processed through the ASIC RTL-to-GDSII flow using Cadence Genus for synthesis, Cadence Conformal for logic equivalence checking, and Cadence Innovus for physical implementation. Timing and power analysis can be used to evaluate the implementation characteristics of the design.

Overall, the project demonstrates the practical implementation of a Hamming SEC ECC system and provides experience in SystemVerilog RTL design, functional verification, error correction techniques, logic synthesis, equivalence checking, physical design, timing analysis, and ASIC implementation methodology.

REFERENCES:

R. W. Hamming, “Error Detecting and Error Correcting Codes,” Bell System Technical Journal, vol. 29, pp. 147–160, Apr. 1950. DOI: 10.1002/j.1538-7305.1950.tb00463.x.

OA Monitor Ireland

“Error Detection and Correction by Hamming Code,” in 2016 International Conference on Global Trends in Signal Processing, Information Computing and Communication (ICGTSPICC), 2016. DOI: 10.1109/ICGTSPICC.2016.7955265.